Arboles

Esta última estructura que vamos a estudiar es de las más complejas y a la vez útiles en informática. Existen muchos tipos de árboles para diversos usos, pero nosotros nos vamos a centrar en el árbol ABB, árbol binario de búsqueda. En su forma más básica, un árbol consiste en nodos conectados por aristas, donde cada nodo tiene un nodo padre (excepto el nodo raíz) y cero o más nodos hijos.

Implementaciones de árboles en informática

- Sistemas de Archivos

Los sistemas operativos utilizan árboles para organizar y gestionar la estructura jerárquica de los archivos en el disco. En muchos sistemas, la estructura de directorios se modela como un árbol, donde:

1. Cada directorio es un nodo que puede tener múltiples subdirectorios o archivos como hijos.
2. El nodo raíz (root) representa el directorio principal del sistema, y cada archivo o carpeta puede ser alcanzado navegando desde este nodo a través de las ramas.
3. Por ejemplo, en un sistema de archivos como el de Linux o Windows, cada directorio tiene subdirectorios y archivos, formando una estructura de árbol.

- Bases de Datos

Los árboles son fundamentales en el diseño de estructuras de bases de datos, en especial en los índices utilizados para acceder a los datos de manera rápida y eficiente. Ejemplos incluyen:

1. Árboles B y B+: Utilizados en bases de datos relacionales para organizar los índices de las tablas y mejorar la velocidad de búsqueda, inserción y eliminación de registros.
2. Árbol AVL y Árbol Rojo-Negro: Estas estructuras balanceadas permiten realizar operaciones en tiempo logarítmico, garantizando una respuesta rápida incluso con grandes volúmenes de datos.

- Inteligencia Artificial y Juegos

En inteligencia artificial, los árboles son una estructura central para tomar decisiones:

1. Árboles de decisión: Usados en aprendizaje automático para modelar decisiones y posibles consecuencias. Son útiles en clasificación, regresión y análisis de datos.
2. Minimax y Alfa-Beta: Algoritmos utilizados para optimizar decisiones en juegos de dos jugadores, como ajedrez o damas, donde cada nodo representa un estado del juego y los nodos hijos representan movimientos posibles.

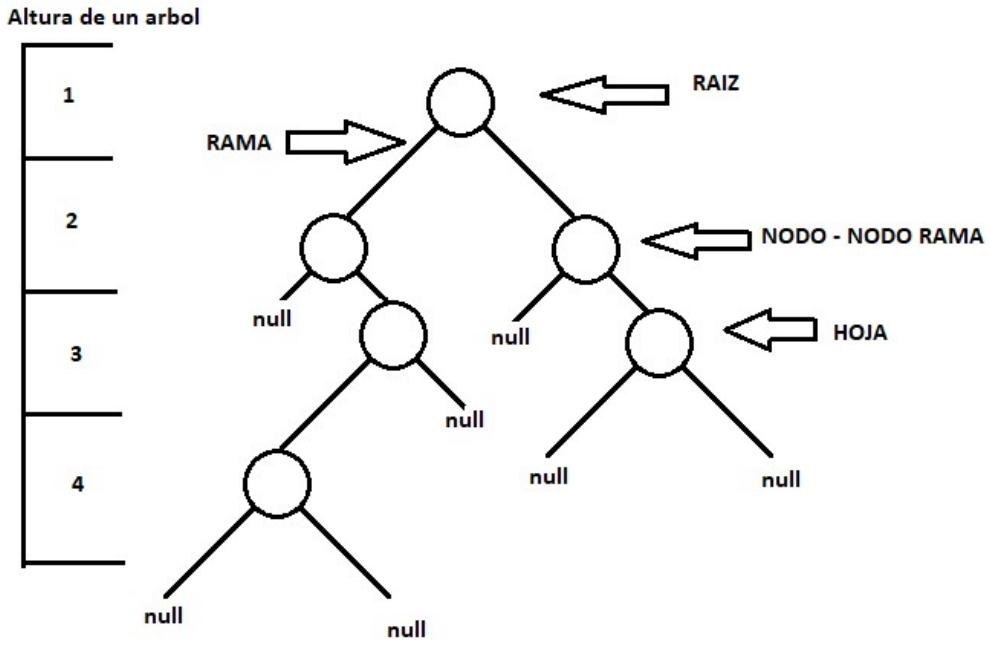
Otros usos más específicos:

- Compiladores y Lenguajes de Programación: Los compiladores utilizan árboles para analizar y procesar el código fuente de los programas.
- Redes y Enrutamiento. En redes, los algoritmos de enrutamiento pueden basarse en árboles para determinar la mejor ruta desde un nodo origen a un nodo destino.
- Compresión de Datos
- Motores de Búsqueda
- Autómatas y Procesamiento de Texto

Componentes de un Árbol

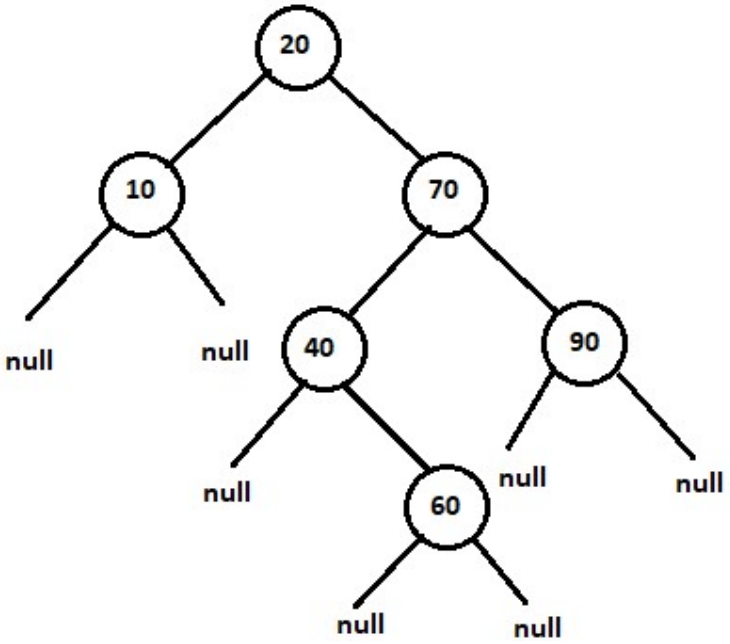
- Nodo: Elemento básico que contiene un dato y referencias a otros nodos.
- Raíz: El nodo superior del árbol que no tiene nodo padre.
- Hoja: Un nodo que no tiene nodos hijos.
- Altura del Árbol: La longitud del camino más largo desde la raíz hasta una hoja.
- Subárbol: Un árbol formado por un nodo y todos sus descendientes.
- Rama: Un nodo que tiene al menos un hijo

Como mencionamos anteriormente los arboles a analizar en este curso son ABB, arboles binarios de búsqueda. Son binarios porque cada nodo tiene como máximo dos ramas y de búsqueda porque queda optimizado para la rápida búsqueda de un dato dentro del árbol. Como en todos los casos anteriores para simplificar su estudio vamos a trabajar con un árbol de enteros.



Para empezar a codificar esta estructura, utilizaremos también un struc y dos punteros internos. El código nos va a ser igual o similar al de la estructura de una lista, pero la diferencia va a radicar en como manipulemos los datos. En principio observemos que en vez anterior y siguiente tenemos izquierda y derecha. Aplicando la lógica para este tipo de estructura vamos a decir que un dato menor al dato actual va a ir hacia la izquierda y un dato mayor al nodo actual va a ir hacia la derecha.

```
typedef struct arbol
{
    int dato;
    struct arbol *izquierda;
    struct arbol *derecha;
}tarbol;
```



Sub árbol

Cada nodo cuenta con una rama izquierda y una rama derecha, cada una de estas ramas puede contener otro nodo y así sucesivamente. Entonces sobre cada rama puede existir otro sub árbol de tal manera que en la rama izquierda hacemos referencia al sub árbol izquierdo y en las ramas derechas de cada nodo hacemos referencia al sub árbol derecho

Vamos a seguir codificando este árbol, la siguiente función que vamos a crear es la de crear un nodo, para luego insertarlo en un árbol.

```
tarbol* crearNodoArbol(int dato)
{
    tarbol *nuevo=(tarbol*)malloc(sizeof(tarbol));
    nuevo->dato=dato;
    nuevo->izquierda=NULL;
    nuevo->derecha=NULL;
}
```

Insertando datos

```
void agregarEnArbol(tarbol * a,int dato)
{
    if(dato<a->dato)
    {
        if(a->izquierda==NULL)
        {
            tarbol *nuevo=crearNodoArbol(dato);
            a->izquierda=nuevo;
        }
        else{
            agregarEnArbol(a->izquierda,dato);
        }
    }
    else{
        if(a->derecha==NULL)
        {
            tarbol *nuevo=crearNodoArbol(dato);
            a->derecha=nuevo;
        }
        else{
            agregarEnArbol(a->derecha,dato);
        }
    }
}
```

Para insertar datos en primer lugar verificamos si el dato a insertar es mayor o menor al dato actual. Para cualquiera de los dos casos elegimos una ruta, siendo menor a la izquierda y siendo mayor a la derecha. Una vez que determinamos eso, debemos ver si esa rama a continuación está libre o está ocupada por otro nodo. Si esta libre se inserta el nodo nuevo con el dato y caso contrario en forma recursiva avanzamos sobre esa rama hasta encontrar un lugar libre. Este mismo principio de avance sobre ramas y recursión es el que vamos a utilizar para recorrer y mostrar el árbol.

Recorriendo y mostrando los datos de un árbol

Los recorridos de un árbol son formas de visitar todos los nodos del árbol. Los tres recorridos principales son:

- Preorden: Raíz → Subárbol izquierdo → Subárbol derecho
- Inorden: Subárbol izquierdo → Raíz → Subárbol derecho
- Postorden: Subárbol izquierdo → Subárbol derecho → Raíz

Podemos destacar que, en el árbol binario de búsqueda, al organizar los datos menores a la izquierda y los mayores a la derecha, nos permite realizar una búsqueda InOrden y mostrar los datos en forma ordenada de mayor a menor, y con las otras dos opciones nos permite encontrar rápidamente un dato en un menor tiempo que en cualquier otra estructura.

In orden

```
void mostrarArbolEnOrden(tarbol *a)
{
    if(a)
    {
        mostrarArbolEnOrden(a->izquierda);
        printf("%d\n",a->dato);
        mostrarArbolEnOrden(a->derecha);
    }
}
```

Esta recorrida primero realiza la recursión sobre los subárboles izquierdos luego muestra los datos en la medida que vuelve de la recursión y luego avanza sobre los subárboles derechos realizando la misma acción. Como este algoritmo primero ingresa totalmente a la izquierda, nos permite mostrar los datos en forma ordenada de mayor a menor.

Pre orden y post orden

```
void mostrarArbolPreOrden(tarbol *a)
{
    if(a)
    {
        printf("%d\n",a->dato);
        mostrarArbolPreOrden(a->izquierda);
        mostrarArbolPreOrden(a->derecha);
    }
}
```

```
void mostrarArbolPostOrden(tarbol *a)
{
    if(a)
    {
        mostrarArbolPostOrden(a->izquierda);
        mostrarArbolPostOrden(a->derecha);
        printf("%d\n",a->dato);
    }
}
```

Para cualquiera de estos dos casos, cambia la posición en donde se muestran los datos, en la izquierda primero muestra los datos luego avanza sobre los sub arboles izquierdos y luego sobre los derechos. Para el caso de la derecha, primero avanza todo a la derecha y luego va mostrando los datos al cerrar la recursión.

Implementamos un programa sencillo

```
int main()
{
    tarbol * nuevoarbol=crearNodoArbol(20);
    agregarEnArbol(nuevoarbol,40);
    agregarEnArbol(nuevoarbol,10);
    agregarEnArbol(nuevoarbol,15);
    agregarEnArbol(nuevoarbol,60);
    agregarEnArbol(nuevoarbol,90);
    agregarEnArbol(nuevoarbol,80);
    printf("Arbol en orden\n");
    mostrarArbolEnOrden(nuevoarbol);
    printf("Arbol en pre orden\n");
    mostrarArbolPreOrden(nuevoarbol);
    printf("Arbol en post orden\n");
    mostrarArbolPostOrden(nuevoarbol);
}
```